DOORS Database Design Document

Data Orchestration and On Demand Reporting System

Logical and Physical Database Baseline for Master and GePNIC Write Databases

Document attribute	Value
Document type	Database Design Document and Data Dictionary
Product baseline	DOORS 1.0.9
Document version	Audit Draft 1.0
Prepared date	25 October 2026
Primary audience	HOG, Data Managers, GePNIC application teams, DBAs, developers, operations, security and audit teams
Implementation evidence	Master API JPA entities, Flyway migrations and governed write-service SQL generator
Classification	Internal project document

Document Control

Version	Date	Owner	Change summary	Status
1.0	25 October 2026	DOORS project team	Initial as-built database design, catalogue, security model and operational guidance	Audit Draft

Approval Record

Role	Name	Decision	Date
Project owner		Pending	
GePNIC technical authority		Pending	
Database authority		Pending	
Security authority		Pending	
Quality assurance lead		Pending	

Contents

1 Purpose and Scope

2 Database Architecture

3 Logical Data Model and ER Diagrams

4 Master Database Catalogue

5 Core Master Data Dictionaries

6 Governed Write Database Design

7 Keys Constraints and Indexing

8 Database Security and Privileges

9 Transactions Concurrency and Idempotency

10 Data Lifecycle Retention and Recovery

11 Schema Change and Deployment

12 Monitoring Capacity and Performance

13 Verification and Acceptance

- Appendix A PostgreSQL Type Guidance
- Appendix B Verification Queries
- Appendix C Source Baseline and Limitations

1 Purpose and Scope

This document defines the logical and physical database design of DOORS. It complements the Software Requirements Specification by describing persistent entities, relationships, governed write structures, constraints, database roles, migrations, integrity controls, recovery expectations and verification evidence.

1.1 Design Baseline

The design is an as-built baseline derived from the DOORS Master API entity mappings, version-controlled Flyway migrations and the JSON write-service SQL generator available on 25 October 2026. The deployed PostgreSQL catalogue remains the authoritative physical record for a specific environment. Differences must be reconciled through an approved migration rather than manual drift.

1.2 Included Databases

Database boundary	Owner	Contents	Write authority
DOORS Master database	DOORS platform DBA	Identity, governance, clients, mappings, policies, audits, subscriptions, AIra history and write-service metadata	Master API runtime through application role; migrations through DBA/deployment role
GePNIC operational database	GePNIC DBA	Existing public/business schema plus DBA-created doors_write schema	Existing GePNIC applications retain their established permissions
doors_write schema	GePNIC DBA	Per-service immutable JSON history, head, receipt, service grant, latest view and store function	DOORS runtime executes approved function only; no runtime DDL or direct table DML
AIra vector store	AI/vector service owner	Approved knowledge chunks, embeddings and source metadata	Indexer and retrieval identities only

1.3 Design Principles

- Database credentials remain outside browser-visible data and generated SDKs.
- Governance data and business data are separated by database boundary and responsibility.
- Approved read services execute stored SELECT definitions through read-only database identities.
- Live write APIs never create or alter destination schema objects.
- Write history is append-only; current state is a derived head and latest view.
- All schema changes are repeatable, version-controlled and auditable.
- JSON payload preservation does not bypass approved field, key or type validation.

2 Database Architecture

2.1 Data Movement

Stage	Database interaction	Security boundary	Persistent result
Human governance	UI calls Master API	RBAC, MFA, plane and CSRF	Master governance and audit tables
Read execution	Master sends approved query and values to Agent	Transit secret, HTTPS and Agent mapping	Result is returned; business result is not copied into Master tables
Write execution	Master sends validated contract request to Agent	Client, service, Agent and database grant intersection	Append-only snapshot and idempotency receipt in doors_write

Stage	Database interaction	Security boundary	Persistent result
Document delivery	Agent resolves authorised document	Eligibility and canonical root	Audit metadata and receipt; document body not stored in Master DB
AIra	Master retrieves approved knowledge or authorised aggregates	Role and data-scope enforcement	Conversation metadata and approved vector knowledge

2.2 PostgreSQL Schemas

Schema	Location	Purpose	Access rule
Application/default schema	Master DB	Fixed DOORS metadata and governance tables	Application role and migration role only
public	GePNIC DB	Existing GePNIC tables and views	doors receives read-only access where required; no structure change by write-service SQL
doors_write	GePNIC DB	Common namespace for approved write-service objects	PUBLIC revoked; DBA owner; function-mediated runtime writes
Vector collection/index	Vector DB	Retrieval index for approved documents	Not a source of live transaction truth

3 Logical Data Model

3.1 Principal Relationships

Parent	Relationship	Child or dependent	Cardinality and rule
users	authorised machine clients	user_api_clients	Many to many; mapping does not itself grant query or write permission
users	authorised Agents	user_authorized_agents	One user to many Agent identifiers
agents	approved query targets	sql_template_authorized_agents	Many to many through query ownership
external_api_clients	approved read services	external_client_query_map	Many clients to many queries with optional segregation filter
sql_templates	schema contracts	template_contracts	One query to versioned contracts
write_contracts	Agent deployments	write_deployments	One approved contract to one or more independently verified Agents
write_deployments	client route	inbound_write_routes	A route references one verified deployment
external_api_clients	write profile	write_client_profiles	One to zero or one environment/application profile
users	subscription	api_user_licenses	One current licence per API user
api_user_licenses	accounting periods	api_subscription_periods	One licence to many independently versioned periods
document policy	service	document_service_registry	Service references a policy code and manifest query/client
aira_conversations	exchanges	aira_chat_exchanges	One conversation to ordered exchanges

3.2 Master Database Logical ER Diagram

This diagram groups the principal Master database entities and their governed relationships. Associative mapping tables implement many-to-many user, client, query and Agent scopes; the catalogue in Section 4 lists the complete physical table inventory.

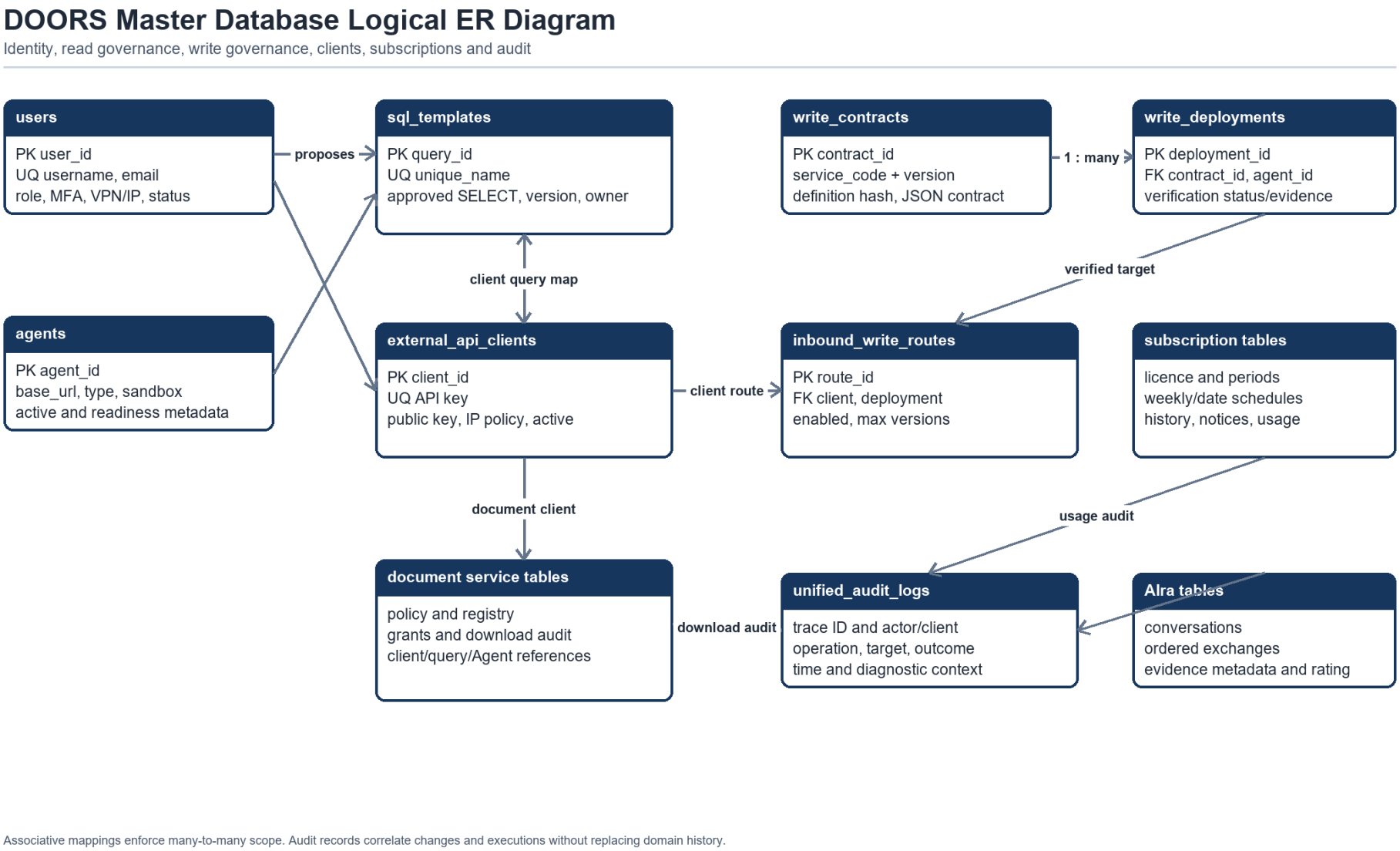


Figure - 3.2 Master Database Logical ER Diagram

3.3 Write Service Database ER Diagram

For each approved write service, the record identity is instance_id plus business_key. Changed JSON appends record_version plus one. The head row serialises version assignment, receipts make retries deterministic, and the latest view exposes the current record. The function is the only runtime mutation boundary; history, head and receipt changes occur in one transaction.

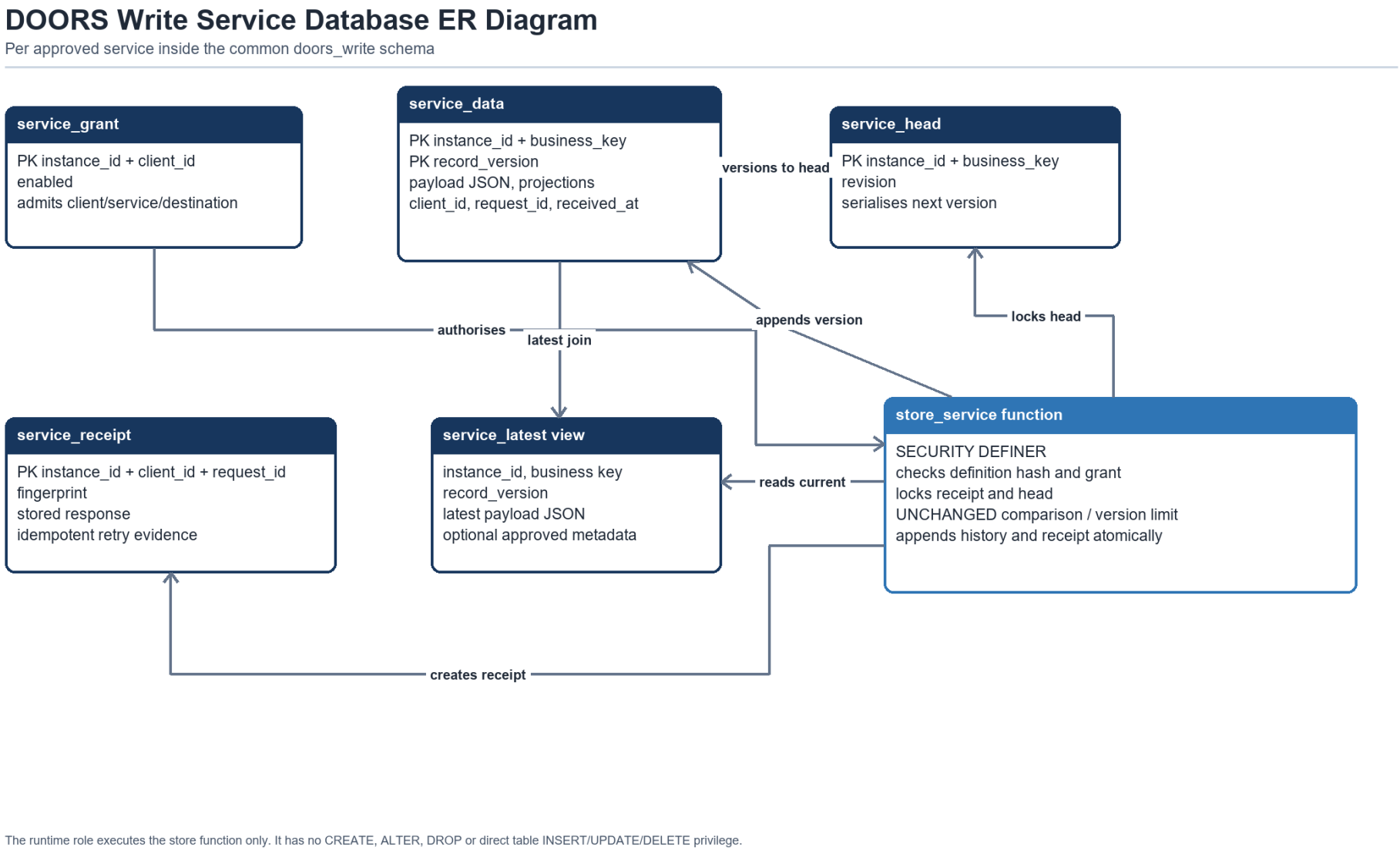


Figure - 3.3 Write Service Database ER Diagram

3.4 Document Service Logical ER Diagram

This diagram shows policy, service routing, eligibility, machine-client identity and download evidence. Documents remain at the governed Agent source; the Master database stores policies, grants and audit metadata rather than document bodies.

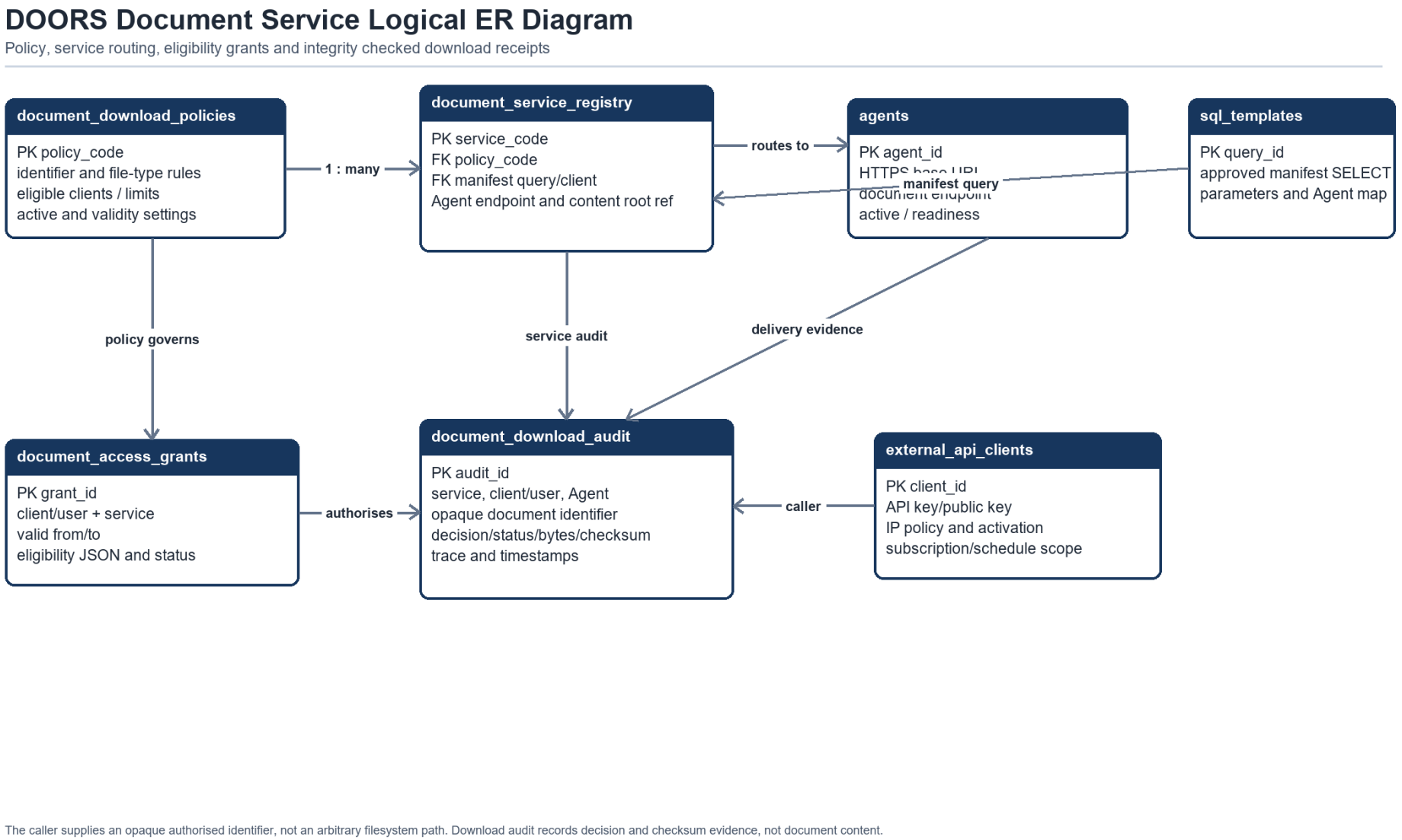


Figure - 3.4 Document Service Logical ER Diagram

4 Master Database Catalogue

Domain	Table	Purpose
Identity	users	Human identity, password hash, role, account status, session, MFA and VPN controls
Identity	user_authorized_agents	User-to-Agent scope
Identity	user_api_clients	User-to-API-client ownership
Infrastructure	agents	Registered Agent endpoint, environment and operational metadata
Read governance	sql_templates	Versioned approved SELECT definitions and test state
Read governance	sql_template_authorized_agents	Query-to-Agent mapping
Read governance	query_categories	Query catalogue hierarchy
Read governance	template_contracts	Versioned request/response JSON schemas
Requests	data_pull_requests	External data request lifecycle and attachments
Clients	external_api_clients	Machine client, API key, IP policy, public key and activation
Clients	external_client_query_map	Client-to-query authorisation and response segregation
Clients	api_keys	Legacy or internal consumer key registry where enabled
Clients	doors_signing_certificates	DOORS signing-key identifiers and lifecycle metadata
Gateway	gateway_client_protocol	Client protocol enrolment
Gateway	gateway_request_claim	Replay/idempotency claim with expiry
Gateway	gateway_protocol_catalog	Supported protocol catalogue
Gateway	gateway_query_protocol	Query protocol activation
Gateway	gateway_protocol_policy_audit	Protocol policy change history
Documents	document_download_policies	Eligibility policy, accepted identifiers, clients and types
Documents	document_service_registry	Agent document endpoint and policy linkage
Documents	document_access_grants	Time-bound grant and eligibility JSON
Documents	document_download_audit	Manifest, decision, stream, checksum, response and trace evidence
Subscriptions	api_user_licenses	Current API-user service and Agent licence
Subscriptions	api_license_history	Versioned licence snapshots
Subscriptions	api_subscription_periods	Independent accounting/access periods
Subscriptions	api_subscription_period_history	Period history
Subscriptions	api_client_schedules	Client date-specific access windows
Subscriptions	api_client_schedule_history	Schedule history
Subscriptions	api_user_weekly_access	Weekly access windows
Subscriptions	api_user_weekly_access_history	Weekly-window history
Subscriptions	api_subscriber_notices	Client notices
Subscriptions	api_subscriber_notice_reads	Per-user acknowledgement
Subscriptions	api_egress_events	Response byte and timing accounting
Subscriptions	api_usage_metadata	Usage tracking baseline
Write governance	write_contracts	Approved JSON contract and definition hash

Domain	Table	Purpose
Write governance	write_deployments	Contract-to-Agent verification
Write governance	write_governance_events	Proposer/reviewer/verification history
Write routing	inbound_write_routes	Client, service and verified deployment route
Write routing	inbound_write_audit	Write admission and outcome audit
Write routing	write_client_profiles	Client environment and application profile
Audit	unified_audit_logs	Cross-feature execution and security audit
Alra	aira_conversations	Conversation ownership and title
Alra	aira_chat_exchanges	Questions, answers, evidence metadata and rating

5 Core Master Data Dictionaries

5.1 users

Human identity and privileged access state. Passwords are stored only as hashes; MFA secrets are encrypted.

Column or object	Type	Null or default	Key or rule	Purpose
user_id	integer	Not null	Primary key, identity	Internal user identifier
username	varchar	Not null	Unique	Login and audit identity
password_hash	varchar	Not null	No plaintext	Adaptive password hash
email	varchar	Not null	Unique	Official contact/login attribute
role	varchar	Not null	Allowed application role	RBAC authority source
is_active, status	boolean, varchar	Defaults controlled	Account lifecycle	Admission state
password_reset_required	boolean	Default true	Forced-change rule	Temporary/reset credential control
name, org, description	varchar/text	Optional	Profile	Human-readable ownership
current_session_id	varchar	Optional	Revocable session binding	Single/current session tracking
mfa_enabled	boolean	Default false	MFA state	Second-factor requirement
mfa_secret_encrypted	text	Optional	Encrypted at rest	TOTP secret ciphertext
vpn_ip	varchar(1024)	Optional	IP/CIDR list	Network admission
vpn_certificate_reference	varchar	Optional	Reference only	VPN evidence reference
vpn_status	varchar(30)	Not null	Policy state	VPN entitlement
privileged_approved_by/at	varchar, timestamp	Optional	Approval evidence	Privileged access authorisation
created_at	timestamp	Database generated	Audit	Creation time

5.2 agents

Master registry for independently deployed Agent APIs. Target passwords should remain in protected Agent configuration rather than ordinary UI-visible values.

Column or object	Type	Null or default	Key or rule	Purpose
agent_id	varchar	Not null	Primary/unique identity	Stable route identifier

Column or object	Type	Null or default	Key or rule	Purpose
display_name	varchar	Not null		User-facing name
base_url	varchar	Not null	HTTPS required outside local dev	Agent API root
agent_instance_code	varchar	Optional		Deployment identifier
agent_type	varchar	Not null	Enum/policy	Environment or purpose
target_db_host/port/name/user	varchar/integer	Optional metadata	Protected display	Diagnostic target metadata
target_db_password	varchar	Legacy/avoid	Secret policy	Must not be exposed; prefer external Agent-side secret
is_active	boolean	Default policy	Admission	Whether Master may route
is_sandbox	boolean	Default false	Sandbox selection	Test-only classification
status	varchar	Optional	Operational state	Health/readiness summary
created/updated fields	timestamp/varchar	Not null	Audit	Actor and time

5.3 sql_templates

Governed reusable SELECT definitions.

Column or object	Type	Null or default	Key or rule	Purpose
query_id	bigint	Not null	Primary key	Query identity
unique_name	varchar(100)	Not null	Unique	Machine/service name
sql_text	text	Not null	SELECT-only security policy	Approved SQL
description	varchar(500)	Optional		Purpose
category, subcategory	varchar	Optional	Catalogue	Classification
status	varchar(20)	Default pending	State model	Governance state
proposer_id, approver_id	varchar(50)	Optional by stage	Separation of duties	Actors
version	integer	Default 1	Revision	Definition version
is_active, is_secure	boolean	Defaults true	Admission/security	Runtime switches
governance_note	text	Optional	Review evidence	Decision rationale
last_tested_agent/result	varchar/text	Optional	Test evidence	Most recent dry run
request_id, submission_source	bigint, varchar	Optional	Origin linkage	External request or internal
parameters	text[]	Default empty	Only :p_ names	Approved input names
created/updated fields	timestamp/varchar	Not null	Inherited audit	Lifecycle evidence

5.4 external_api_clients

Machine identity registry used by read and write services.

Column or object	Type	Null or default	Key or rule	Purpose
client_id	bigint	Not null	Primary key	Client identity
client_name	varchar	Not null		Application name

Column or object	Type	Null or default	Key or rule	Purpose
api_key	varchar	Not null	Unique; protected display	Passport credential
allowed_ips	string/collection	Optional	CIDR/IP policy	Network admission
is_active	boolean	Default policy	Fail closed	Client admission
client_public_key	text	Optional until configured	RSA/certificate validation	Signature verification and response encryption
is_encryption_enabled	boolean	Default policy	Protocol control	Application-layer encryption
created_by/at, last_used_at	varchar/timestamp	Audited		Lifecycle and use evidence

5.5 external_client_query_map

Independent read permission joining a client to a governed query.

Column or object	Type	Null or default	Key or rule	Purpose
client_id	bigint	Not null	Composite identity/FK	Authorised client
query_id	bigint	Not null	Composite identity/FK	Authorised query
assigned_by/at	varchar/timestamp	Not null by service	Audit	Grant actor/time
response_filter_column	varchar	Optional	Approved returned column	Segregation field
response_filter_value	varchar	Optional	Bound client value	Segregation value
response_filter_fields	text/json per migration	Optional	Allowlist	Configurable response filtering

5.6 unified_audit_logs

Cross-service audit and execution evidence. It must not store passwords, private keys, AES keys or raw authentication credentials.

Column or object	Type	Null or default	Key or rule	Purpose
log_id	bigint	Not null	Primary key	Audit identity
event_type	varchar	Not null	Controlled vocabulary	Event classification
username	varchar	Optional for machine/system events	Actor	Human principal
query_id, query_name	bigint/varchar	Optional	Governed object	Read context
endpoint, method	varchar	Optional	Request context	Route and verb
status_code	integer	Optional	HTTP result	Outcome
duration_ms	bigint	Optional	Non-negative	Latency
client_ip	varchar	Optional	Policy evidence	Source address
error_message, error_code	text/varchar(100)	Optional	Safe diagnostic	Failure evidence
record_count	bigint	Optional	Non-negative	Returned count
trace_id	varchar(50)	Optional	Indexed/correlated	Cross-service trace
execution_time	timestamp	DB generated	Chronology	Event time

5.7 write_contracts

Master definition of a governed JSON write service.

Column or object	Type	Null or default	Key or rule	Purpose
id	bigserial	Not null	Primary key	Contract identity
service_code	varchar(40)	Not null	Lowercase pattern; version-unique	Stable service name
contract_version	integer	Not null	Unique with service_code	JSON contract revision
title, purpose	varchar/text	Not null	Validated	Human description
sample_json	text	Not null	Representative only	Approved example
definition_hash	char(64)	Not null	SHA-256	DBA/migration integrity
definition_json	jsonb	Not null	Canonical contract	Fields, types, keys and rules
status	varchar(20)	Not null	Governed state	Draft/pending/approved/rejected/disabled as deployed
proposer, reviewer, review_note	text	Stage dependent	No self-approval	Transaction history
request_id	bigint	Optional	Origin	Approved request linkage
sandbox_agent/evidence/tested_at	varchar/jsonb/timestamptz	Optional until test	Sandbox gate	Test evidence
category/subcategory	varchar(80)	Defaults	Catalogue	Classification
created_at/reviewed_at	timestamptz	Lifecycle		Chronology

5.8 write_deployments and inbound_write_routes

Separates contract approval, Agent verification and client permission.

Column or object	Type	Null or default	Key or rule	Purpose
write_deployments.id	bigserial	Not null	Primary key	Deployment identity
contract_id	bigint	Not null	FK write_contracts	Approved contract
instance_id	varchar(80)	Not null	Unique with contract in current schema	Stored destination context
agent_id	varchar(255)	Not null	FK agents	Destination Agent
status	varchar(20)	Default NOT_PROVISIONED	Verified only after checks	Readiness
verification, verified_at	jsonb/timestamptz	Optional	Evidence	Schema and permission result
inbound_write_routes.id	bigserial	Not null	Primary key	Client route
service_code, instance_id	varchar	Not null	Business route	Write service context
client_id	bigint	Not null	FK external_api_clients	Authorised caller
deployment_id	bigint	Required for governed route	FK write_deployments	Verified target
enabled	boolean	Default false	Fail closed	Runtime switch
protocol_version	integer	Default 1	Supported policy	Transport contract
max_versions	bigint	Optional	1..1,000,000	Per-client changed-snapshot limit
agent_routing	boolean	Default false	Migration compatibility	Agent-only routing mode

Column or object	Type	Null or default	Key or rule	Purpose
created_by/at	text/timestamptz	Not null	Audit	Grant actor/time

5.9 Supporting Master Tables

Table group	Important keys and controls	Design notes
template_contracts	contract_id PK; unique(template_id, contract_version); schema_hash; lock_version	Versioned request/response schema with optimistic locking
data_pull_requests	request_id PK; requested_by; status; approved_by; query_id	External request and approval evidence; attachments are controlled
document_download_policies	policy_id PK; policy_code unique; JSONB identifiers/types/clients; policy_version; lock_version	Eligibility definition is versioned and deny-by-default
document_service_registry	service_id PK; Agent; base/download URL; manifest query/client; policy; TLS/timeouts	No arbitrary caller-supplied server path
document_download_audit	audit id; client/service/correlation/trace; outcome/status/checksums	No document body stored
gateway_request_claim	request identifier/claim and expiry	Replay/freshness boundary
api_user_licenses	user_id PK/FK; agent_id FK; service_enabled; version	Current machine-service entitlement
api_subscription_periods	period_id PK; user/date unique; amounts non-negative; version	Accounting periods independent from current licence
api_client_schedules and weekly access	client/user keys; JSON/date or day/time windows; history	Asia/Kolkata unless configuration says otherwise
api_egress_events	event_id PK; client; endpoint/status/bytes/count/duration/time	Stores metrics, not response payload
aira_conversations/exchanges	conversation and exchange PK/FK; question/answer; evidence metadata; rating	Role-scoped conversation history

6 Governed Write Database Design

The SQL generator derives object names from the approved service code. In the common schema the generic pattern below is used; for the Agreement service the names are agreement_data, agreement_data_head, agreement_data_receipt, agreement_data_service_grant, agreement_data_latest and store_agreement_data. Generated SQL must be reviewed against the approved definition hash before execution.

6.1 service data history table

Append-only business snapshots. Additional approved scalar projections are service-specific columns; payload retains the complete received JSON.

Column or object	Type	Null or default	Key or rule	Purpose
instance_id	text	Not null	PK part; validated pattern	Destination/consolidation context
business_key	jsonb	Not null	PK part	Ordered approved record-key values
record_version	bigint	Not null	PK part; >0	Automatic immutable version
payload	json	Not null	Object check	Complete received document
client_id	bigint	Not null	Audit/source	Submitting client
request_id	uuid	Not null	Receipt-linked	Logical submission ID

Column or object	Type	Null or default	Key or rule	Purpose
received_at	timestamptz	Default clock_timestamp	Chronology	Database receipt time
approved projections	PostgreSQL type	Contract-specific	Validated before insert	Optional readable/scalar columns

6.2 service head table

Serialises version allocation and identifies the current record.

Column or object	Type	Null or default	Key or rule	Purpose
instance_id	text	Not null	Primary key part	Destination context
business_key	jsonb	Not null	Primary key part	Record identity
revision	bigint	Default 0	Row locked FOR UPDATE	Latest stored version

6.3 service receipt table

Makes write retries idempotent and preserves the original response.

Column or object	Type	Null or default	Key or rule	Purpose
instance_id	text	Not null	Primary key part	Destination context
client_id	bigint	Not null	Primary key part	Caller
request_id	uuid	Not null	Primary key part	Idempotency key
fingerprint	text	Not null	SHA-256 lowercase hex	Canonical logical request fingerprint
response	jsonb	Not null	Stored outcome	Replay response source

6.4 service grant table

Database-side defence requiring an explicit client and instance grant.

Column or object	Type	Null or default	Key or rule	Purpose
instance_id	text	Not null	Primary key part	Destination context
client_id	bigint	Not null	Primary key part	Authorised client
enabled	boolean	Default false	Must be true	Runtime function admission

6.5 latest view

Read model for GePNIC consumers. It joins history to head on instance_id and business_key and exposes the row whose record_version equals revision.

Column or object	Type	Null or default	Key or rule	Purpose
business key columns	service-specific	Not null	Filter	Tender/agreement or equivalent identity
instance_id	text	Available	Required for consolidated/multi-instance data	Destination context
record_version	bigint	Not null	Latest only	Current snapshot number
payload	json	Not null	Primary consumer value	Latest complete JSON

Column or object	Type	Null or default	Key or rule	Purpose
metadata/projections	service-specific	As approved	Read-only	Optional operational columns

6.6 Store Function Contract

Parameter	Type	Validation and purpose
p_instance	text	Required; approved identifier; selects service grant and stored context
p_client	bigint	Required positive registered client
p_request	uuid	Required; exact retry uses same value
p_expected	bigint	Compatibility parameter; automatic-version contracts use server-managed semantics
p_fingerprint	text	Required SHA-256 of logical request
p_payload	json	Required JSON object; contract and type validated
p_definition	text	Must equal approved 64-character definition hash

6.7 Store Function Algorithm

1. Validate definition hash, metadata, instance, request identifier, fingerprint, payload object and record-key values.

2. Confirm enabled service_grant for instance_id and client_id.

3. Acquire an advisory transaction lock for request identity and inspect receipt.

4. Return the stored response with replayed=true when request and fingerprint match; reject changed content using the same request ID.

5. Create or find head and lock the business-key row FOR UPDATE.

6. Canonicalise/compare the latest payload for automatic-version services; return UNCHANGED without increment when identical.

7. Enforce the client-wise version limit before storing changed JSON.

8. Validate and convert approved projected fields, insert the next immutable version, and update head atomically.

9. Insert the receipt and return JSON containing outcome, replay status, requestId, instanceId and recordVersion.

7 Keys Constraints and Indexing

Area	Required key or constraint	Reason
Identity	Unique users.username and users.email	Prevents duplicate principals
Clients	Unique API key; client_id PK	Unambiguous machine identity
Queries	Unique sql_templates.unique_name	Stable API service name
Contracts	Unique service_code + contract_version; definition hash	Immutable governed revision
Deployments	Contract/Agent or contract/instance uniqueness according to current migration	Prevents ambiguous target verification
Read mapping	Client + query uniqueness	No duplicate authorisation
Write route	Service + destination + client uniqueness	One effective route per client and service context
Write history	instance_id + business_key + record_version PK	No duplicate snapshot version
Write receipt	instance_id + client_id + request_id PK	Idempotency

Area	Required key or constraint	Reason
Subscription	User/date-period uniqueness and non-negative amount checks	Accounting integrity
Audit	Indexes on trace ID, error code, client/time and outcome/time	Investigation performance

7.1 Index Review

Before production, EXPLAIN plans and observed workloads shall confirm indexes for login identifiers, active clients, Agent mappings, query/service lookup, trace/time audit searches, subscription windows, write business keys and latest-view joins. Indexes shall not be added merely to every column; write amplification and retention volume must be measured.

8 Database Security and Privileges

Role	Master database	GePNIC public schema	doors_write schema
Master application role	CRUD only on required Master objects; no superuser or schema ownership	No direct connection	No direct connection
Flyway/deployment role	Apply approved Master migrations	None	None
Agent read role	None	USAGE plus SELECT on approved/existing objects	Read latest views only if explicitly granted
doors runtime role	None	USAGE and SELECT where combined read/write Agent requires it	USAGE plus EXECUTE on approved store function; SELECT on latest view; no CREATE, ALTER, DROP or direct table DML
GePNIC application role such as gepuser	None	Existing application permissions	USAGE and SELECT on approved latest/read views; no direct write
GePNIC DBA/schema owner	None	Existing DBA authority	Owns schema, tables, views and SECURITY DEFINER functions; executes approved SQL
PUBLIC	No application grant	Environment policy	Revoked on doors_write schema, tables, views and functions

8.1 SECURITY DEFINER Requirements

- Function owner is the approved DBA/schema owner, not the doors runtime role.
- search_path is fixed to pg_catalog and doors_write to prevent object-shadowing attacks.
- PUBLIC EXECUTE is revoked and only the runtime role receives EXECUTE.
- Inputs are validated before dynamic effects; generated identifiers are derived from approved definitions, not caller text.
- A lock timeout and deterministic DateStyle, TimeZone and monetary/interval settings are applied.
- Verification checks owner, prosecdef, function signature, proconfig and effective runtime privileges.

9 Transactions Concurrency and Idempotency

Concern	Mechanism	Failure behaviour
Concurrent updates to one business key	Head row FOR UPDATE inside one database transaction	Exactly one next record_version; loser observes updated state
Repeated exact request	Receipt primary key and stored fingerprint/response	Returns replayed outcome without a second snapshot
Changed payload with same request ID	Fingerprint comparison	IDEMPOTENCY_CONFLICT; no data mutation

Concern	Mechanism	Failure behaviour
Identical JSON with new request ID	Canonical semantic comparison	UNCHANGED; no record-version increment
Version limit	Client route max_versions checked before insert	Conflict with clear administrator contact instruction
Timeout after downstream invocation	Caller retries same request ID and identical payload	Receipt resolves STORED/UNCHANGED versus unknown
Migration error	DDL/provisioning transaction	Rollback; existing data retained

10 Data Lifecycle Retention and Recovery

Data class	Lifecycle characteristic	Recovery requirement
Identity and governance	Retain active state and immutable decision/revision history per approved policy	Restore relationships, roles and approval history consistently
Audit and execution	Time-partition/archive when volume requires; legal hold overrides disposal	Trace IDs and chronology remain intact
Write history	Immutable business snapshots; disposal only under business/legal authority	Restore history, head and receipts from the same recovery point
Write receipts	Retain at least as long as supported retry/idempotency window and audit need	Never restore a head without matching committed version
Documents	File content remains at governed source; Master stores metadata/receipt only	Reconcile source availability and receipt evidence
AIra	Conversation retention follows privacy policy; vector index is rebuildable from approved sources	Restore source baseline or reindex after validated recovery
Subscriptions	Current state plus versioned history and periods	No silent loss of accounting or access history

10.1 Backup Consistency

Master and GePNIC databases can have independent backup schedules, but a write-service incident may require correlated restore points. Restore testing shall validate schema version, row counts, foreign keys, write head-to-history consistency, receipt replay, latest views, permissions and application smoke tests. RPO, RTO, backup encryption and retention values are approved per environment in the SRS policy register.

11 Schema Change and Deployment

Change type	Required method	Preconditions	Rollback
Master metadata	Versioned Flyway migration	Backup, checksum, test and compatible application build	Forward fix or tested database restore
New write service	Approved downloadable DBA SQL	Contract approval, hash verification and role prerequisites	Transaction rollback; remove only newly created empty objects under DBA approval
Write contract revision	Generated migration SQL	Approved revision, sandbox evidence, data compatibility and verified hash	Rollback transaction or restore; never discard existing versions
Index/performance change	DBA-reviewed migration	EXPLAIN evidence and lock/space assessment	Drop/revert index through approved change
Privilege change	Explicit GRANT/REVOKE script	Role inventory and least-privilege review	Restore previous grants from recorded baseline

11.1 Deployment Order

1. Back up affected database and export current schema/privilege baseline.
2. Verify artefact and migration SHA-256 values and confirm the target environment.
3. Apply Master migrations before starting a Master build that depends on them.
4. Deploy Agent only when its database routes, validation, verification or write behaviour changed.
5. Have the GePNIC DBA run approved write-service SQL in the mapped database.
6. Run DOORS schema-and-permission verification against every mapped Agent.
7. Run read, write, document, audit and unaffected-service smoke tests.
8. Record migration version, executor, start/end time, result and rollback decision.

12 Monitoring Capacity and Performance

Monitor	Indicators	Action threshold or response
Connections	Active/idle connections, pool wait and exhaustion	Tune pool within DB capacity; identify leaks
Queries	Latency, timeout, rows, temporary files and blocked sessions	Review SQL/index and preserve bounded execution
Locks	Waiters, deadlocks and long transactions	Correlate trace and terminate only under DBA SOP
Storage	Table/index growth, WAL, backup size and free space	Forecast write history and audit retention
Autovacuum	Dead tuples, last vacuum/analyse and wraparound risk	Tune high-churn audit/subscription tables
Write integrity	Head without version, receipt conflicts, version-limit events	Investigate before further writes
Replication/backup	Lag, failed backup and restore-test age	Escalate against RPO/RTO
Privileges	Unexpected owner/grantee or PUBLIC access	Revoke, investigate drift and rerun verification

13 Verification and Acceptance

Gate	Minimum evidence
Inventory	Deployed schema catalogue reconciled with entities and Flyway history
Integrity	All PK, FK, unique, check and not-null constraints valid
Security	Role matrix proven with positive and negative GRANT tests; PUBLIC revoked from doors_write
Write function	Owner, SECURITY DEFINER, fixed settings, signature, grant, STORED, UNCHANGED, replay and conflict tests
Read service	Read-only account and bounded approved query execution
Migration	Upgrade from current staging copy preserves data and permissions
Recovery	Restore rehearsal verifies Master relations and write head/history/receipt consistency
Performance	Representative EXPLAIN and load evidence within approved limits
Audit	Trace, client, request, Agent, outcome and time correlation

13.1 Acceptance Responsibility

Authority	Accepts
GePNIC DBA	Physical schema, owners, functions, grants, backup and recovery

Authority	Accepts
DOORS Data Manager	Governance relationships, mappings and operational usability
Security authority	Least privilege, secrets, cryptography-related metadata, audit and retention controls
Application owner	Read/write contract and latest-view consumption
Quality assurance	Migration, regression and negative test evidence

Appendix A PostgreSQL Type Guidance

Contract concept	Preferred PostgreSQL type	Guidance
Identifier	text or varchar(n)	Do not use numeric for values with leading zeros
Date	date	ISO YYYY-MM-DD at API boundary
Timestamp	timestamptz	Store an absolute instant; state timezone conversion explicitly
Money/decimal	numeric(p,s)	Approve precision and scale; avoid floating point
Boolean	boolean	Accept only contract-compatible true/false representation
Whole number	integer or bigint	Select bounds based on business maximum
JSON object/array	jsonb for searchable metadata; json when exact payload preservation is required	Payload design currently preserves received JSON and uses canonical comparison separately
UUID	uuid	Request identifiers and stable opaque IDs
Enumerated value	varchar plus CHECK or reference table	Use a reference table when values change operationally
Large binary	External document store preferred	Avoid Master database BLOB unless specifically approved

Appendix B Verification Queries

The following query purposes shall be implemented in the approved DBA runbook. Literal service object names must be taken from the approved generated SQL.

Verification	Catalogue source	Expected result
Schema owner	pg_namespace joined to pg_roles	doors_write owner is the designated GePNIC DBA role
Objects and types	pg_class/pg_namespace	Expected service tables and latest view exist
Function signature and owner	pg_proc/pg_namespace	Exactly the approved store function; DBA owner; prosecdef=true
Function settings	pg_proc.proconfig	Fixed search_path, lock_timeout, DateStyle, TimeZone, IntervalStyle and monetary locale as generated
Runtime grants	information_schema.role_*_grants and has_*_privilege	doors has schema usage and function execute, but no direct table insert/update/delete or DDL
Reader grants	has_schema_privilege and has_table_privilege	GePNIC application role can select approved latest view
PUBLIC exposure	aclexplode or has_*_privilege for public	No unintended doors_write access
Head consistency	LEFT JOIN head to history on key/revision	Zero missing latest rows
Receipt replay	Controlled duplicate request test	Same stored response and no new version
Latest read	Filter latest view by approved business key	One current JSON snapshot

Appendix C Source Baseline and Limitations

This document records the intended and source-controlled design. It does not substitute for an environment catalogue export. Some early baseline tables predate the visible Flyway sequence and are represented from current JPA mappings. Some generated write columns and object names vary by approved contract. Before audit submission, each environment shall attach a schema-only catalogue, Flyway history, role/grant export and variance statement signed by the DBA.

Evidence source	Design use
Master API JPA entities	Baseline fixed table/column mappings and relationships
Master Flyway migrations	Authoritative incremental DDL, indexes, constraints and history tables
JsonWriteDefinition generator	Authoritative pattern for doors_write objects and store functions
Approved downloadable write SQL	Authoritative service-specific physical DDL and definition hash
PostgreSQL system catalogues	Authoritative deployed owners, grants, constraints, indexes and function settings
SRS and operational SOP	Policy values, retention, recovery and control ownership